

XCS221 Assignment 3 — Route Planning

Due Sunday, September 20 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs221-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset $\text{\LaTeX}$ submission. If you wish to typeset your submission and are new to $\text{\LaTeX}$, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- **hidden:** These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden: Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                           in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

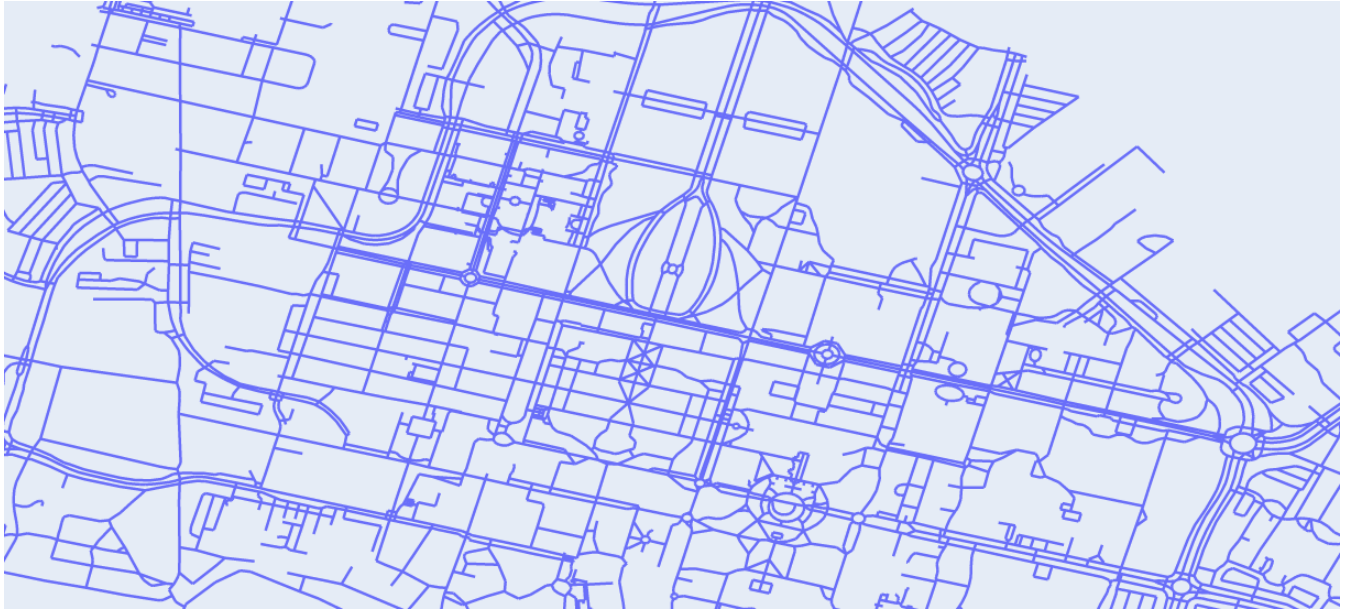
Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

Introduction



In route planning, the objective is to find the best way to get from point A to point B (think Google Maps). In this homework, we will build on top of the classic shortest path problem to allow for more powerful queries. For example, not only will you be able to explicitly ask for the shortest path from the Gates Building to Coupa Cafe at Green Library, but you can ask for the shortest path from Gates back to your dorm, stopping by the package center, gym, and the dining hall (in any order!) along the way.

We will assume that we have a *map* of a city (e.g., Stanford) consisting of a set of *locations*. Each location has:

1. a unique label (e.g., 6608996258),
2. a (latitude, longitude) pair specifying where the location is (e.g., 37.4299866, -122.175519), and
3. a set of *tags* which describes the type of location (e.g., amenity=food).

There are a set of *connections* between pairs of locations; each connection has a *distance* (in meters), and can be traversed in both directions (if the distance from A to B is 100 meters, then the distance from B to A is also 100 meters).

There are two city maps that you'll be working with: a grid map (`createGridMap`) and a map of Stanford (`createStanfordMap`), which is derived from [Open Street Maps](#). Please note that you do not need to know the Stanford campus in order to answer the questions below! We have also included instructions on how to create your own maps in `README.md`.

Important: On Windows some of the commands will lead to the python process not exiting and hanging. If you experience this behaviour, simply kill the process as the results will not be affected!

1. Grid City

Consider an infinite city consisting of locations (x, y) where x, y are integers. From each location (x, y) , one can go east, west, north, or south. You start at $(0, 0)$ and want to go to (m, n) , where $m, n \geq 0$. We can define the following search problem to capture this:

- $s_{\text{start}} = (0, 0)$
- $\text{Actions}(s) = \{(+1, 0), (-1, 0), (0, +1), (0, -1)\}$
- $\text{Succ}(s, a) = s + a$
- $\text{Cost}((x, y), a) = 1 + \max(x, 0)$ (it is more expensive as x increases)
- $\text{IsEnd}(s) = \mathbf{1}[s = (m, n)]$

(a) [1 point (Written)]

What is the minimum cost of reaching location (m, n) (note that $m, n \geq 0$) starting from location $(0, 0)$ in the above city? Describe one possible path achieving the minimum cost. Is it unique (i.e., are there multiple paths that achieve the minimum cost)?

What we expect: Provide an expression for the minimum cost. In addition, using 1 - 2 sentences describe one possible minimum cost path and state whether it is unique.

The minimum cost of reaching (m, n) from $(0, 0)$ is

$$\frac{m(m+1)}{2} + n.$$

The cost only depends on the x being left, so take all n north steps while still at $x = 0$, where each costs 1 and they total n , then the m east steps leave from $x = 0, 1, \dots, m-1$ in turn and cost $1, 2, \dots, m$; writing that series forwards and backwards pairs it into m columns of $m+1$, so it sums to $m(m+1)/2$. One path achieving the minimum is to take all n steps north from the origin first and then all m steps east, and it is the only one, since any north step taken after moving east costs more than 1 because it leaves from $x > 0$, and a detour into negative x adds at least one step west and one step back east, each costing at least 1, without making any other step cheaper.

(b) [1.50 points (Written)] How will Uniform Cost Search (UCS) behave on this problem? Mark the following as true or false:

- UCS will never terminate because the number of states is infinite.
- UCS will return the minimum cost path and explore only locations between $(0, 0)$ and (m, n) ; that is, (x, y) such that $0 \leq x \leq m$ and $0 \leq y \leq n$.
- UCS will return the minimum cost path and explore only locations whose past costs are less than the minimum cost from $(0, 0)$ to (m, n) .

What we expect: T/F for each subpart.

- False. The state space is infinite, but UCS processes states in order of past cost and every step costs at least 1, so a state with past cost below the goal's cost is within that many steps of the origin. There are only finitely many such states, so UCS reaches the goal and terminates.
- False. UCS returns the minimum-cost path, but it explores outside the rectangle: $(-1, 0)$ and $(0, -1)$ each cost 1 to reach, which is cheaper than any goal beyond the origin, so they are processed even though they lie outside $0 \leq x \leq m, 0 \leq y \leq n$.
- False. UCS does return the minimum-cost path, but it can explore locations whose past cost *equals* the minimum rather than being strictly less than it. When several states tie for the lowest cost, the queue's tie-break decides the order. For the goal $(1, 0)$, which costs 1, the states $(-1, 0)$, $(0, -1)$ and $(0, 1)$ also have past cost 1 and are popped before the goal.

(c) [1.50 points (Written)] Now consider running UCS on an arbitrary graph. Mark the following as true or false:

- i. If you add a connection between two locations, the minimum distance cannot go up.
- ii. If you make the cost of an action from some state small enough (possibly negative), that action will show up in the minimum cost path.
- iii. If you increase the cost of each action by 1, the minimum cost path does not change (even though its cost does).

What we expect: T/F for each subpart.

- i. True. Adding a connection only adds paths and never removes one, so the minimum over the larger set of paths cannot exceed the minimum over the smaller set. The distance either drops, if the best path through the new connection is cheaper, or stays the same.
- ii. False. An action out of a state that the start cannot reach lies on no path from the start to the goal, however small its cost. For example, with $A \xrightarrow{5} C$ as the only route from start A to goal C and a separate component $D \xrightarrow{-1000} E$, the cheap action never appears in the minimum-cost path. The same holds for an action leading into a dead end.
- iii. False. Adding 1 to every action raises a path's cost by its number of edges, so paths with more edges are penalised more and the ranking can change. Take route 1, $A \xrightarrow{10} Z$ with one edge, and route 2, $A \xrightarrow{2} B \xrightarrow{2} C \xrightarrow{2} D \xrightarrow{2} Z$ with four. Before the change route 1 costs 10 and route 2 costs 8, so route 2 is the minimum. After adding 1 to every action, route 1 costs 11 and route 2 costs 12, so route 1 becomes the minimum-cost path.

2. Finding Shortest Paths

We first start out with the problem of finding the shortest path from a *start location* (e.g., the Gates Building) to some end location. In Google Maps, you can only specify a specific end location (e.g., Coupa Cafe at Green Library).

In this problem, we want to give the user the flexibility of specifying *multiple* possible end locations by specifying a set of "tags" (e.g., so you can say that you want to go to *any* place with food versus a specific location like Tressider).

(a) [8 points (Coding)]

Implement `ShortestPathProblem` so that given a `startLocation` and `endTag`, the minimum cost path corresponds to the shortest path from `startLocation` to *any* location that has the `endTag`.

In particular, you need to implement `startState()`, `isEnd(state)`, `successorsAndCosts(state)`.

Recall the separation between search problem (modeling) and search algorithm (inference). You should focus on modeling (defining the `ShortestPathProblem`); the default search algorithm, `UniformCostSearch` (UCS), is implemented for you in `util.py`.

What we expect: An implementation of the `ShortestPathProblem` class.

(b) [1 point (Written)]

Run `python mapUtil.py > readableStanfordMap.txt` to write a (long-ish) file of the possible locations on the Stanford map along with their tags. Each tag is a `[key]=[value]`. Here are some examples of keys:

- **landmark:** Hand-defined landmarks (from `src/data/stanford-landmarks.json`)
- **amenity:** Various amenity types (e.g., "park", "food")
- **parking:** Assorted parking options (e.g., "underground")

Choose a starting location and end tag (perhaps that's relevant to your daily life) and implement `getStanfordShortestPathProblem()` to create a search problem. Then, run `python grader.py 2b-0-helper` to generate `path.json`. Once generated, run `python visualization.py` to visualize it (opens in browser). Try different start locations and end tags. Pick two settings corresponding to the following:

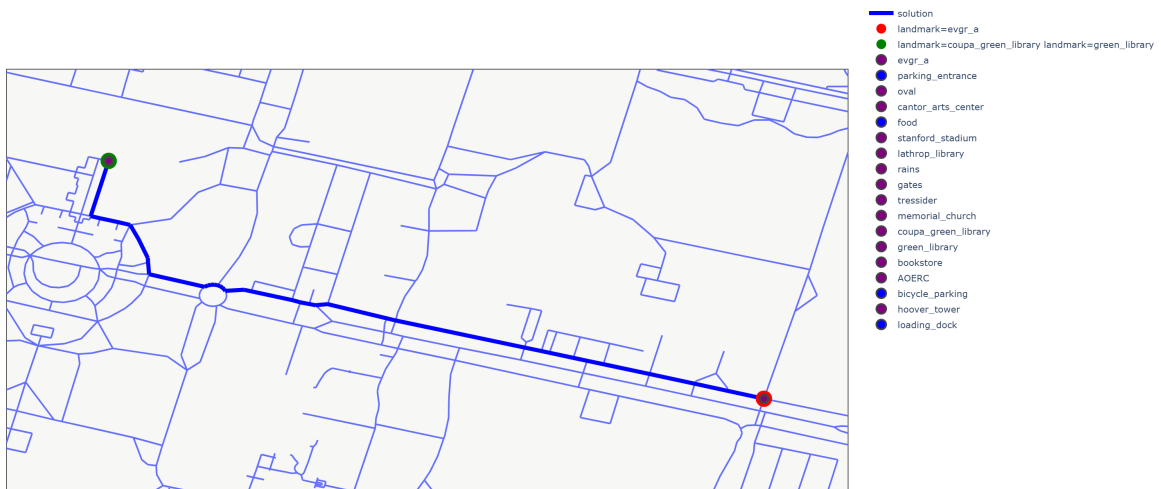
- A start location and end tag that produced new insight into traveling around campus. Describe whether the system was useful.
- A start location and end tag where the minimum cost path found isn't desirable. Is this due to incorrect modeling assumptions? Explain.

You should feel free to add new landmarks and if you are not at Stanford, follow the instructions in the `README.md` to use your own map and landmarks.

What we expect: A screenshot of the visualization of your two solutions as well as i) one or more sentences describing something interesting you've learned about traveling (around campus, or elsewhere) and ii) something incorrect about either the map or modeling assumptions (such a landmark being out of place, etc.).

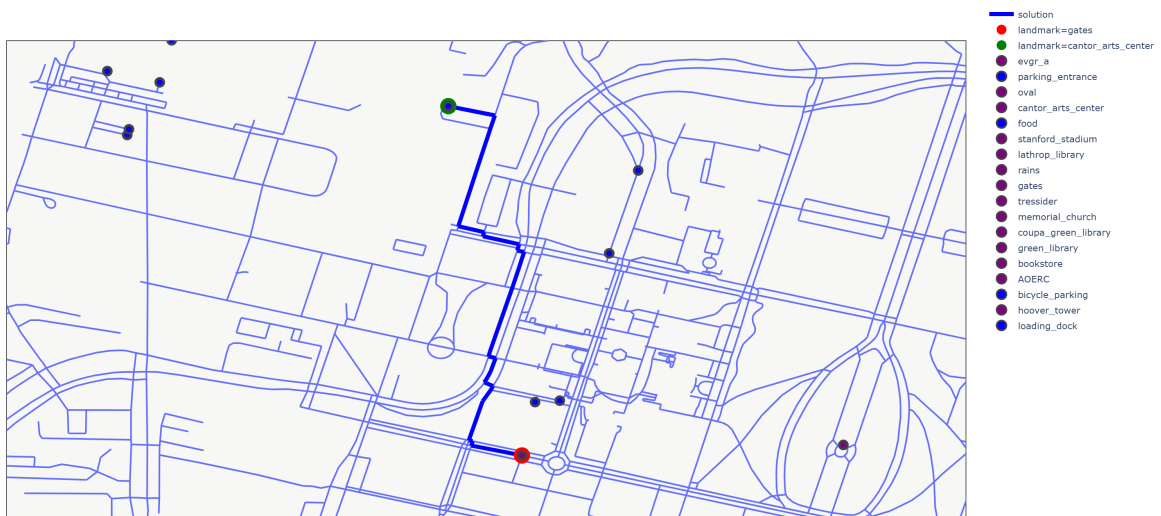
The first setting is the useful one: start at EVGR-A with end tag `landmark=green_library`, which comes out at 606.0 m over 38 locations.

2b (useful): EVGR-A to Green Library



The second is the one where the answer is not desirable: start at Gates with end tag `amenity=food`, 659.2 m, finishing at the Cantor Arts Center.

2b (undesirable): Gates to the nearest amenity=food



What I learned is that the cost is pure distance, so everything that actually slows a walk down is free. My EVGR-A to Green Library route is 606.0 m and passes 2 marked crossings, while the Gates route is almost the same length at 659.2 m but passes 10 crossings and a bus stop, so in practice the second walk takes noticeably longer and the search has no way to tell the two apart. I also found the locations are spaced very unevenly, from 1.85 m to 98.8 m apart on the same path, so the number of steps in a path says nothing about how far it is.

What is wrong is that asking Gates for the nearest `amenity=food` returns the Cantor Arts Center at 659.2 m, while Tressider, which is the actual food court, sits at 965.9 m and never gets picked. Only four locations on the entire map carry `amenity=food`, and two of them are the art museum and the football stadium, because the tag is

attached to the landmark's own OSM node, so one cafe inside a large building ends up labelling the whole building as food. The modelling assumption that breaks is that a tag on a single node stands for the whole place, and that a place I can reach is a place that is open and serving.

(c) [1 point (Written)]

Your system now allows anyone to find the shortest path between any pair of locations on campus. By shortening their travel distance, it promises to optimize travel efficiency. But, there might be issues when a large fraction of the population start using your system to plan their routes.

In particular, what *negative externalities* might result from this system being widely deployed? Please refer to these brief articles for inspiration: "[Why Traffic Apps Make Congestion Worse](#)", "[The Perfect Selfishness of Mapping Apps](#)" as well as the module on Externalities ([video](#) and [pdf](#)).

Discuss the impact of these externalities on (1) users of your system and (2) non-users. Remember that these sorts of problems often arise from the mismatch between the real world and one's model of it. What are potential ways you could reduce this mismatch?

What we expect: Provide 3-5 sentences describing (a) two externalities (one for users of the route-planning system and one for other people/non-users of the system), as well as (b) at least one potential solution to reduce the impact of these problems.

If everyone plans with the same system, the users all get sent down the same shortest path, and the congestion that creates never shows up in the cost, because the cost is only distance. The people who do not use it are hit whenever their own route intersects that optimal path, so quiet side streets that were never built for through traffic end up carrying it, and they get none of the benefit since they never opted in. The mismatch is that the model prices an edge as a fixed number when the real price depends on how many people are on it at the time, and the system has no way of knowing that its own advice is what put them there. One way to reduce that is to put actual speed into the cost rather than distance alone, so a road that is slow or already saturated costs more and the system spreads people out instead of funnelling them all onto one route.

3. Finding Shortest Paths with Unordered Waypoints

Let's introduce an even more powerful feature: unordered waypoints! In Google Maps, you can specify an ordered sequence of waypoints that a path must go through – for example, going from point A to point X to point Y to point B, where [X, Y] are "waypoints" (such as a gas station or a friend's house).

However, we want to consider the case where the waypoints are *unordered*: X, Y , so that both $A \rightarrow X \rightarrow Y \rightarrow B$ and $A \rightarrow Y \rightarrow X \rightarrow B$ are allowed. Moreover, X , Y , and B are each specified by a tag like in Problem 2 (e.g., `amenity=food`)

This is a neat feature if you think about your day-to-day life; you might be on your way home after a long day, but need to stop by the package center, Tressider to grab a bite of food, and the bookstore to buy some notebooks. Having the ability to get a short, quick path that hits all these stops might be really convenient (rather than searching over the various waypoint orderings yourself).

(a) [11 points (Coding)]

Implement `WaypointsShortestPathProblem` so that given a `startLocation`, set of `waypointTags`, and an `endTag`, the minimum cost path corresponds to the shortest path from `startLocation` to a location with the `endTag`, such that all of `waypointTags` are covered by some location in the path.

Note that a single location can be used to satisfy multiple tags.

Like in Problem 2, you need to implement `startState()`, `isEnd(state)`, and `successorsAndCosts(state)`.

There are many ways to implement this search problem, so you should think carefully about how to design your `State`. We want to optimize for a compact state space so that search is as efficient as possible.

What we expect: An implementation of the `WaypointsShortestPathProblem` class. To get full credit, your implementation must have the right asymptotic dependence on the number of locations and waypoints. Note that your code will timeout if you do this incorrectly.

(b) [1.50 points (Written)]

If there are n locations and k waypoint tags, what is the maximum number of states that UCS could visit?

What we expect: A mathematical expression that depends on n and k , with a brief explanation justifying it.

The maximum number of states UCS could visit is

$$n \cdot 2^k.$$

A state is a location together with the set of waypoint tags covered so far, so there are n choices for the location, and for the memory each of the k tags is independently either covered or not, which gives 2^k possible sets. Multiplying the two independent choices gives $n \cdot 2^k$, and it is 2^k rather than $k!$ because the memory is stored as a sorted tuple, so covering two waypoints in either order ends up as the same state.

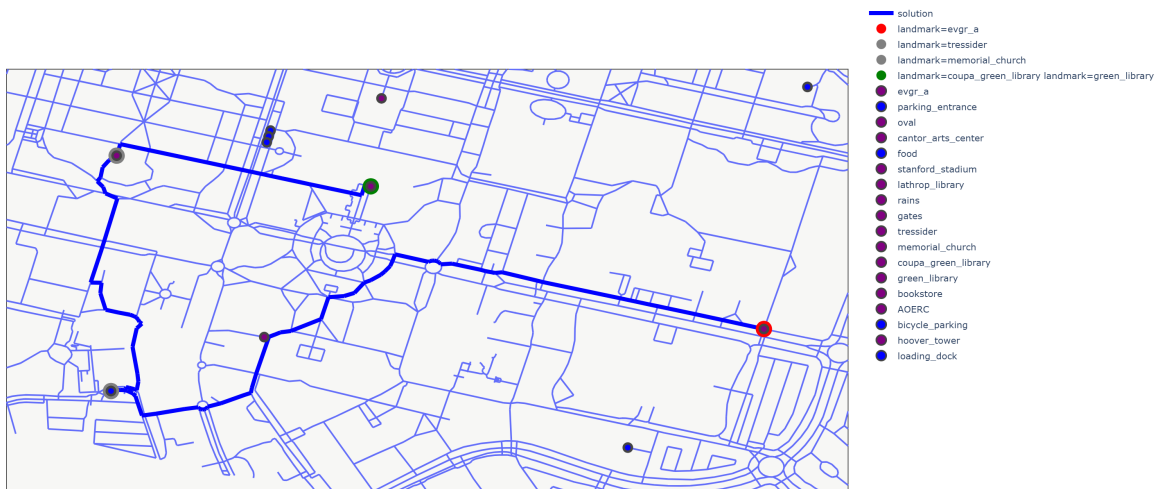
(c) [2 points (Written)]

Choose a starting location, set of waypoint tags, and an end tag (perhaps that captures an interesting route planning problem relevant to you), and implement `getStanfordWaypointsShortestPathProblem()` to create a search problem. Then, similar to Problem 1b, run `python grader.py 3c-0-helper` to generate `path.json`. Once generated, run `python visualization.py` to visualize it (opens in browser).

What we expect: A screenshot of the visualized path with a list of the waypoint tags you selected. In addition, provide one or more sentences describing unexpected or interesting features of the selected path. Does it match your expectations? What does this path fail to capture that might be important?

The waypoint tags are `landmark=memorial_church` and `landmark=tressider`, starting at EVGR-A with end tag `landmark=green_library`, and the path comes out at 1815.6 m over 130 locations.

3c: EVGR-A, via Memorial Church and Tressider, to Green Library



The order surprised me. I listed the church first, but the search went to Tressider at 991.5 m, then the church at 1464.8 m, then the library at 1815.6 m, which is the point of leaving the waypoints unordered: I do not have to guess the ordering, and here my guess would have been the more expensive one. What did match my expectation is the cost, since the same trip with no waypoints is 606.0 m, so adding two stops tripled it, which is what you get when the stops sit off the straight line between start and end.

What the path fails to capture is anything other than metres. It has no idea that Tressider closes in the evening, that the church is worth more time than a junction, or that I would rather eat before settling into the library than after, and it is one way, so it never accounts for getting home. A route that is optimal in distance can still be the wrong plan for the day.

- (d) [1.50 points (Written)] Ride sharing companies use route-finding systems similar to the one you built in this problem to route their drivers. However, these systems have been criticized for using exploitative behavioral nudges. For example, a [New York Times](#) article reported how one app uses “forward dispatch,” a feature that dispatches a new ride to a driver before the current one ends, to constantly keep drivers busy. This makes it more difficult for drivers to take breaks.

We want you to discuss how these companies could implement better labor practices, finding a way to use the unordering waypoints feature to balance company goals with the protection of driver interests. Specifically, we want you to think about ways your new unordered waypoints feature could help drivers in their working experience.

What waypoints could you include from one drop-off location to the next pick-up location to improve the driver experience? What information about drivers would you need to determine appropriate waypoints?

Note, however, that the unordered waypoints feature is an example of a dual use technology. For more, refer to the module on dual use technologies to help answer this question ([video](#) and [pdf](#)). Thus, we also want you to answer: What are some ways that a company could use this waypoints feature to increase profits at the expense of driver health? What are any downsides in collecting the appropriate information you need to recommend the waypoints?

What we expect: Provide 3-5 sentences describing (a) the waypoints you would include to improve the driver experience, (b) at least one dimension of drivers’ identity that could inform selection of waypoints, and (c) a potential negative use of the waypoints feature that a company could use to increase profits while harming the driver experience or driver health, or downsides in collecting the appropriate information needed to recommend the waypoints.

Between a drop-off and the next pick-up I would insert a rest waypoint, tagged for a restroom, food, or somewhere

to park for ten minutes, in the same way problem 4c joins a fake node to every end location at zero cost so the search finds whichever one is nearest. Here the cost would be small rather than zero, so a break is cheap enough that the route takes one but not so free that the driver is sent miles out of the way for it. To pick sensible waypoints I would want the driver's age and years of driving experience, since a new driver and a driver in their seventies do not need the same break pattern or the same length of shift. Those same two fields are exactly what a company could turn against them: knowing a driver's age lets dispatch quietly stop sending work to older drivers, which is age discrimination dressed up as routing, and it never has to be written down anywhere because it only shows up as fewer jobs.

4. Speeding up Search with A*

In this problem, we will explore how to speed up search by reducing the number of states that need to be expanded using various A* heuristics.

(a) [4 points (Coding)]

In this problem, you will implement A* to speed up the search. Recall that in class, A* is just UCS with a different cost function (a new search problem), so we will implement A* via a *reduction* to UCS.

In particular, you should implement `aStarReduction()` which takes a `search problem` and a `heuristic` as input and returns a new search problem.

What we expect: An implementation of the `NewSearchProblem` class in the `aStarReduction(problem, heuristic)` function. As in prior problems, you need to implement `startState()`, `isEnd(state)`, and `successorsAndCosts(state)`.

(b) [6 points (Coding)]

We are now ready to speed up search by simply implementing various heuristics. First, implement `StraightLineHeuristic` for Problem 2, which returns the straight line distance from a state to any location with the end tag. Note: you might want to precompute some things so that evaluating the heuristic is fast.

What we expect: An implementation of the `StraightLineHeuristic` class.

(c) [7 points (Coding)]

Next, implement `NoWaypointsHeuristic` for Problem 3, which returns the minimum cost path from a state to any location with the end tag but ignoring all the waypoints (essentially the solution to Problem 2, so you can reuse that if you'd like). Note: you might want to precompute some things so that evaluating the heuristic is fast. Helpful comments are provided in the code.

What we expect: An implementation of the `NoWaypointsHeuristic` class.

5. AI Tutor

In this problem, you'll participate in an interactive tutoring session with an AI chatbot. The sessions focus on dynamic programming and beam search. By the end, you should be able to explain and apply the basic principles of both techniques, understanding how they help AI systems make efficient and informed decisions.

We recommend using the below prompt template:

Prompt Template

INSERT PROMPT HERE. Keep the session around 15–20 minutes and interactive.

Guidelines

- Adjust difficulty: if I miss 2 in a row → slow + simpler; if I get 2 quickly → a bit harder or add a twist.
- Use tiny, deterministic examples (small numbers, short tables, brief ASCII diagrams, code blocks). Show actual *run-throughs* (step-by-step states) if helps.
- Use Code if it clarifies: ≤ 2 Python/NumPy blocks, each ≤ 20 lines; ideally no imports beyond `import numpy as np`.
- Do NOT solve graded work. If I paste any, refuse and make a similar practice item.
- Be CONCISE. Don't make notation complicated/unnecessary. Focus on intuitions. Code blocks should be short with simple variable names.
- I may interrupt and ask to start somewhere else. Simply adjust and adapt from there.

Suggested flow (flexible; use judgment)

- Start by explaining the topic in a few simple, but technically accurate sentences (explain core idea (≤ 6 sentences): intuition + minimal notation + why it matters + one common pitfall). Use basic notations and visualizations if necessary (CS221 requires basic math/programming backgrounds). For topics about programming, OK to start with concise code blocks.
- Quick check-in (≤ 1 –2 min): ask 2 short questions about prior exposure and specific goal.
- Exercises: use a tiny instance and show intermediate states. Give exercises that build on each other. OK to give multiple exercises at the same time.
 - e.g., for graph search: a 4–5 node weighted graph; show the frontier/priority-queue after each of 2–3 steps and which edges relax.
 - for DP/MDP/RL/Logic/ML, do an equivalently small, concrete step-by-step trace.
 - If code clarifies, include one concise NumPy block.
- 3 quick checks: increasing difficulty; give a hint first; reveal answers only after I try or ask.
- Assessment and feedback: Give me feedback on my performance - what I understood well, what needs improvement, and specific concepts to review further.
- Session reflection: Ask me for feedback on how the tutoring session went - what worked well, what could be improved, and if the pacing/difficulty was appropriate.
- Recap: a 60-second summary

(a) [0.50 points (Written)] Practice Dynamic Programming with an AI tutor!

Use an AI chatbot (e.g., ChatGPT, Claude, Gemini).

For the prompt, use **"Teach me basic Dynamic Programming interactively"**. We recommend using the template provided above.

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be around 15–20 minutes and interactive!

(b) [0.50 points (Written)] Practice Beam Search with an AI tutor!

Use an AI chatbot (e.g., ChatGPT, Claude, Gemini).

For the prompt, use **"Teach me basic Beam Search interactively"**. We recommend using the template

provided above.

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be around 15–20 minutes and interactive!

6. Product Ethics Exploration: Documentation, Marketing, and Customer Support

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it. Please note that the questions regarding your chosen product are interconnected and will progressively build upon each other from one assignment to the next.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product's official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will examine how the company communicates with users through marketing, documentation, and support channels.

Why it's important: The gap between marketing promises and actual capabilities, as well as the quality of user support, directly impacts user trust and the product's real-world effectiveness.

Suggested resources:

- Company marketing pages, ads, YouTube demos
- Social media promotions (LinkedIn, TikTok, Instagram, X)
- Reddit, StackOverflow, Discord, or unofficial user communities
- Product FAQ, Help Center
- Customer service emails/chat logs you submit

(a) [0.50 points (Written)] Marketing Channels and Discovery

- How is the product marketed and how do users discover it? Describe the channels and messaging the company uses.

What we expect: In 4-6 sentences, provide a summary of the main marketing channels and discovery avenues for the product, with links, cited examples, and/or screenshots.

(b) [0.50 points (Written)] Marketing Strategy and Mission Alignment

- Summarize what you believe the company's **core mission** is, based on official statements (About page, press releases, etc.).
- Discuss whether the product's marketing strategy and messaging **align or conflict with the company's mission**. In your view, does the marketing sensationalize or misrepresent what the product can do, or is it fair and accurate? Provide rationale and cite examples (ads, taglines, posts, etc.).

What we expect: In 4-6 sentences, clearly articulate whether the company's marketing supports, contradicts, or sensationalizes its stated mission and product capabilities, providing specific evidence.

(c) [0.50 points (Written)] Feedback, Help, and Customer Support

- How can users contact the company or get support? Describe the official support channels, any community-based help resources, and your overall impression of how easy it is to get assistance.

What we expect: In 2-3 sentences, summarize the avenues available for support and community help, and comment on the effectiveness of official and unofficial documentation resources.

(d) [0.50 points (Written)] Submitting Customer Support Queries

- Submit **two customer support queries** based on your personal experience using the product. These should reflect genuine questions or typical issues you encountered. List those queries here and reflect in 1-2 sentences how customer support responded to and handled your queries.

What we expect: Include two well-constructed sample support emails or chat messages, each clearly stating a question, problem, or request for help and, for each query, 1-2 sentences reflecting on how the company handled that query.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

The minimum cost of reaching (m, n) from $(0, 0)$ is

$$\frac{m(m+1)}{2} + n.$$

The cost only depends on the x being left, so take all n north steps while still at $x = 0$, where each costs 1 and they total n , then the m east steps leave from $x = 0, 1, \dots, m-1$ in turn and cost $1, 2, \dots, m$; writing that series forwards and backwards pairs it into m columns of $m+1$, so it sums to $m(m+1)/2$. One path achieving the minimum is to take all n steps north from the origin first and then all m steps east, and it is the only one, since any north step taken after moving east costs more than 1 because it leaves from $x > 0$, and a detour into negative x adds at least one step west and one step back east, each costing at least 1, without making any other step cheaper.

1.b

1. False. The state space is infinite, but UCS processes states in order of past cost and every step costs at least 1, so a state with past cost below the goal's cost is within that many steps of the origin. There are only finitely many such states, so UCS reaches the goal and terminates.
2. False. UCS returns the minimum-cost path, but it explores outside the rectangle: $(-1, 0)$ and $(0, -1)$ each cost 1 to reach, which is cheaper than any goal beyond the origin, so they are processed even though they lie outside $0 \leq x \leq m, 0 \leq y \leq n$.
3. False. UCS does return the minimum-cost path, but it can explore locations whose past cost *equals* the minimum rather than being strictly less than it. When several states tie for the lowest cost, the queue's tie-break decides the order. For the goal $(1, 0)$, which costs 1, the states $(-1, 0)$, $(0, -1)$ and $(0, 1)$ also have past cost 1 and are popped before the goal.

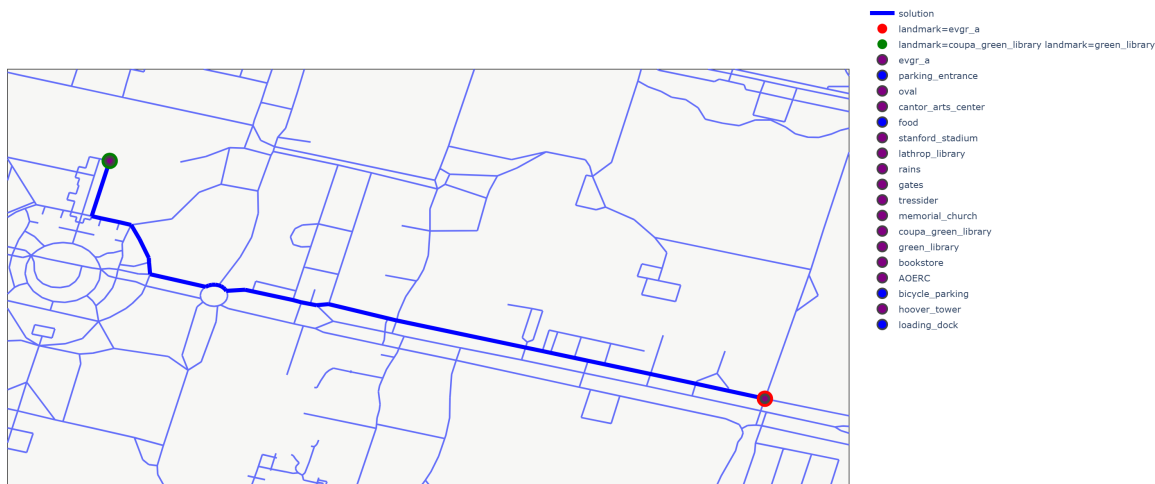
1.c

1. True. Adding a connection only adds paths and never removes one, so the minimum over the larger set of paths cannot exceed the minimum over the smaller set. The distance either drops, if the best path through the new connection is cheaper, or stays the same.
2. False. An action out of a state that the start cannot reach lies on no path from the start to the goal, however small its cost. For example, with $A \xrightarrow{5} C$ as the only route from start A to goal C and a separate component $D \xrightarrow{-1000} E$, the cheap action never appears in the minimum-cost path. The same holds for an action leading into a dead end.
3. False. Adding 1 to every action raises a path's cost by its number of edges, so paths with more edges are penalised more and the ranking can change. Take route 1, $A \xrightarrow{10} Z$ with one edge, and route 2, $A \xrightarrow{2} B \xrightarrow{2} C \xrightarrow{2} D \xrightarrow{2} Z$ with four. Before the change route 1 costs 10 and route 2 costs 8, so route 2 is the minimum. After adding 1 to every action, route 1 costs 11 and route 2 costs 12, so route 1 becomes the minimum-cost path.

2.b

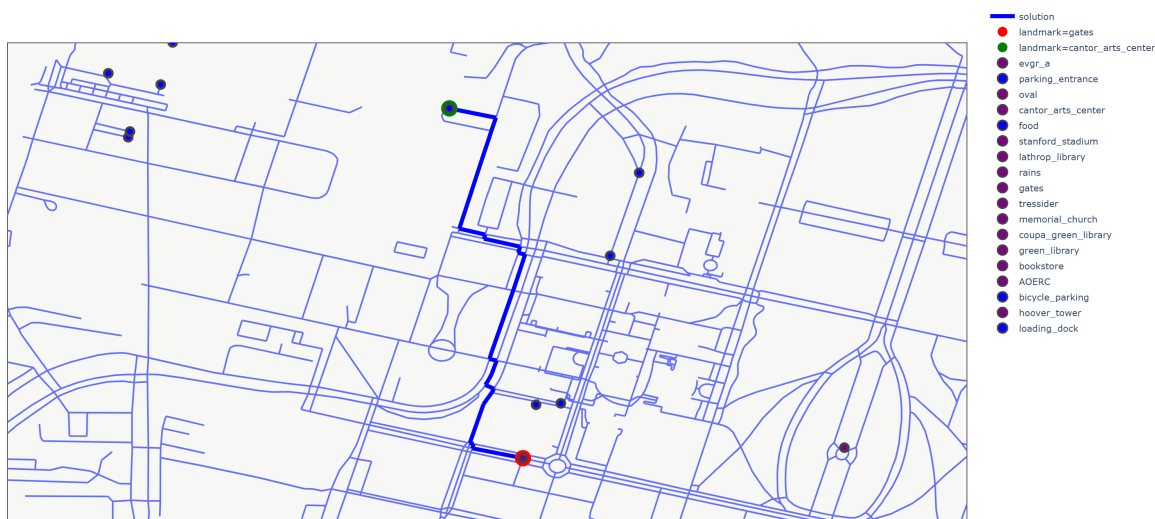
The first setting is the useful one: start at EVGR-A with end tag `landmark=green_library`, which comes out at 606.0 m over 38 locations.

2b (useful): EVGR-A to Green Library



The second is the one where the answer is not desirable: start at Gates with end tag `amenity=food`, 659.2 m, finishing at the Cantor Arts Center.

2b (undesirable): Gates to the nearest amenity=food



What I learned is that the cost is pure distance, so everything that actually slows a walk down is free. My EVGR-A to Green Library route is 606.0 m and passes 2 marked crossings, while the Gates route is almost the same length at 659.2 m but passes 10 crossings and a bus stop, so in practice the second walk takes noticeably longer and the search has no

way to tell the two apart. I also found the locations are spaced very unevenly, from 1.85 m to 98.8 m apart on the same path, so the number of steps in a path says nothing about how far it is.

What is wrong is that asking Gates for the nearest `amenity=food` returns the Cantor Arts Center at 659.2 m, while Tressider, which is the actual food court, sits at 965.9 m and never gets picked. Only four locations on the entire map carry `amenity=food`, and two of them are the art museum and the football stadium, because the tag is attached to the landmark's own OSM node, so one cafe inside a large building ends up labelling the whole building as food. The modelling assumption that breaks is that a tag on a single node stands for the whole place, and that a place I can reach is a place that is open and serving.

2.c

If everyone plans with the same system, the users all get sent down the same shortest path, and the congestion that creates never shows up in the cost, because the cost is only distance. The people who do not use it are hit whenever their own route intersects that optimal path, so quiet side streets that were never built for through traffic end up carrying it, and they get none of the benefit since they never opted in. The mismatch is that the model prices an edge as a fixed number when the real price depends on how many people are on it at the time, and the system has no way of knowing that its own advice is what put them there. One way to reduce that is to put actual speed into the cost rather than distance alone, so a road that is slow or already saturated costs more and the system spreads people out instead of funnelling them all onto one route.

3.b

The maximum number of states UCS could visit is

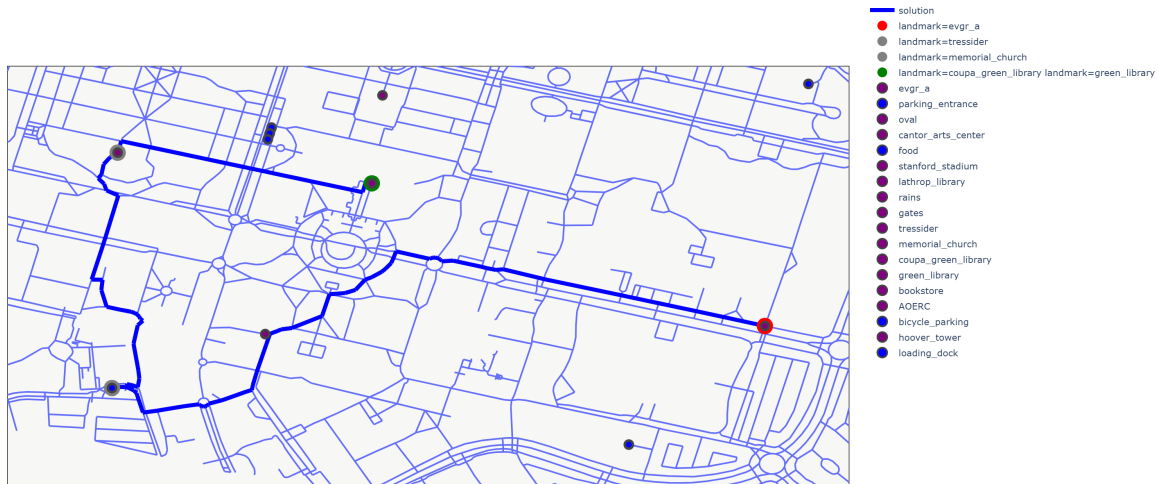
$$n \cdot 2^k.$$

A state is a location together with the set of waypoint tags covered so far, so there are n choices for the location, and for the memory each of the k tags is independently either covered or not, which gives 2^k possible sets. Multiplying the two independent choices gives $n \cdot 2^k$, and it is 2^k rather than $k!$ because the memory is stored as a sorted tuple, so covering two waypoints in either order ends up as the same state.

3.c

The waypoint tags are `landmark=memorial_church` and `landmark=tressider`, starting at EVGR-A with end tag `landmark=green_library`, and the path comes out at 1815.6 m over 130 locations.

3c: EVGR-A, via Memorial Church and Tressider, to Green Library



The order surprised me. I listed the church first, but the search went to Tressider at 991.5 m, then the church at 1464.8 m, then the library at 1815.6 m, which is the point of leaving the waypoints unordered: I do not have to guess the ordering, and here my guess would have been the more expensive one. What did match my expectation is the cost, since the same trip with no waypoints is 606.0 m, so adding two stops tripled it, which is what you get when the stops sit off the straight line between start and end.

What the path fails to capture is anything other than metres. It has no idea that Tressider closes in the evening, that the church is worth more time than a junction, or that I would rather eat before settling into the library than after, and it is one way, so it never accounts for getting home. A route that is optimal in distance can still be the wrong plan for the day.

3.d

Between a drop-off and the next pick-up I would insert a rest waypoint, tagged for a restroom, food, or somewhere to park for ten minutes, in the same way problem 4c joins a fake node to every end location at zero cost so the search finds whichever one is nearest. Here the cost would be small rather than zero, so a break is cheap enough that the route takes one but not so free that the driver is sent miles out of the way for it. To pick sensible waypoints I would want the driver's age and years of driving experience, since a new driver and a driver in their seventies do not need the same break pattern or the same length of shift. Those same two fields are exactly what a company could turn against them: knowing a driver's age lets dispatch quietly stop sending work to older drivers, which is age discrimination dressed up as routing, and it never has to be written down anywhere because it only shows up as fewer jobs.

5.a

5.b

- 6.a
- 6.b
- 6.c
- 6.d